AI Bootcamp — Overview

4-Week Readiness Program for Embedded Engineers

© 2026 Improving

PURPOSE

This program prepares Senior/Principal-level engineers to serve as Embedded Engineers in Deep Learning Program (DLP) engagements. Candidates enter at **Basic Prompting** (AI present but not structurally embedded in their work) and must exit at **DLP Completion** before their first engagement.

This is a prerequisite gate, not a development track. The goal is not gradual improvement. It is achieving full DLP Completion in 4 weeks of intensive, structured work, using the same artifact formats, vocabulary, and evaluation discipline as the DLP v2 program they will go on to deliver.

© 2026 Improving

CANDIDATE PROFILE

Who enters this program:

- Senior or Principal-level developers
- Strong coaching skills and presence
- Not yet at DLP Completion: they use AI tools off-the-shelf but have not yet built skills, written evaluation scripts, or changed their default workflow behaviors

What they arrive with (pre-requisite, not covered here):

- A license for at least one IDE-native agentic coding tool: Claude Code, Devin, or GitHub Copilot
- That tool installed, authenticated, and working on their primary machine
- Familiarity with their tool's basic interface (chat, inline suggestions, skill invocation)

What they must demonstrate to pass:

- Can explain the Prompt/Harness/Model framework and apply the RTCC prompt structure (Role, Task, Context, Constraints) to real work
- Can write Evaluation Criteria (EC) and manual test files (Input Prompts + Expected Outputs) before building any skill
- Can direct an AI to write Python evaluation scripts (deterministic checks plus LLM judges) and interpret the results. There is no Python prerequisite; the models carry the coding load, the candidate directs and evaluates.
- Can apply the EDD Red/Green/Refactor discipline: every instruction in a skill is load-bearing, with a failing evaluation to prove it
- Can craft skills with `disable-model-invocation: true`, graduate one to model invocation when the evidence supports it, and build an adversarial agent that challenges another skill's output
- Can run a bounded agent: a multi-step pipeline chained in a single IDE session with evaluation at the checkpoints
- Has changed at least one real personal workflow to use their new skill as the default behavior
- Can articulate what DLP Completion looks like and coach someone else toward it

© 2026 Improving

FOUNDATIONAL CONCEPTS

PROMPT/HARNESS/MODEL

The conceptual frame for everything in this program:

- **Prompt**: the instructions you give to a model. What you want it to do, how you want it done, what constraints it must follow.
- **Harness**: the software environment that mediates between you and the model. The harness determines what the model can actually do: read files, run commands, access tools. Claude Code, Devin, GitHub Copilot, and ChatGPT web are all different harnesses with different boundaries.
- **Model**: the AI itself. The model generates text based on its training and the prompt, but it cannot do anything on your machine that the harness does not allow.

The harness is the capability boundary; the model is the intelligence; the prompt is the instruction. A model without a harness is a chat interface. This is why skills carry `disable-model-invocation: true`: invocation is a harness-level decision, not something the prompt asks the model for.

EVALUATION-DRIVEN DEVELOPMENT (EDD)

EDD is the most important discipline in this program. Every skill, every evaluation, every workflow is built test-first.

The Rule:

> *Before writing a prompt, write the test. Define what success looks like, specifically, in writing, before you try to produce it.*

EDD Red/Green/Refactor for Skills:

- **Red**: write your Evaluation Criteria (EC) and manual test cases (Input Prompts + Expected Outputs) by hand. Do not use AI to generate them. Run the current skill against them (or have your evaluation script do it). Some checks should fail.
- **Green**: add instructions to the skill until all checks pass.
- **Refactor**: review every instruction. If removing it does not cause any check to fail, remove it. Dead instructions are noise.

The Load-Bearing Rule:

> *If your test would pass without the instruction you're about to add, do not add it.*

This is the analogue of TDD's Red/Green/Refactor. A prompt instruction that causes no check to fail is not doing anything, or worse, it is telling the model something it was already going to do, crowding out instructions that matter.

Why manual test files for skills that alter behavior: When you write a test file manually, you are committing to what success looks like before the model has influenced your expectations. If the model would have produced the output anyway, the instruction isn't altering behavior. It is just narrating what was already going to happen. Manually written test files prevent this: you define the target, then close the gap.

© 2026 Improving

How evaluation is measured: with Python evaluation scripts that the AI writes and you direct and review. Deterministic checks (schema, counts, format) come first: they are fast, free, and definitive. LLM judges (structured JSON verdicts with per-criterion reasoning) are for criteria that genuinely require qualitative judgment. If you find yourself using an LLM judge for something that could be a regex, your EC needs refinement.

THE AUTOMATION SPECTRUM

```
Skills (invocation disabled)
  → Skills (invocation enabled)
    → Adversarial Agents (AI challenges AI)
      → Bounded Agents (multi-step orchestration in the IDE)
        → Custom Harnesses (programmatic orchestration; demoed, not built here)
```

Each step extends what you can trust AI to do, and each step is earned through evaluation evidence. This program operates at IIOS Stage 3 (Task). Stage 4 (Workflow: custom harnesses, automated guardrails, unattended runs) is roughly a 10x effort jump requiring team coordination and IT infrastructure, and it is out of scope. A Stage 4 workflow built on weak Stage 3 foundations is a slop cannon.

© 2026 Improving

STRUCTURE

Week	Focus	Days
1	Foundations: EDD discipline, Prompt/Harness/Model, RTCC, skills via Social Media Content Creation	Half Mon + Tue–Thu full + Half Fri
2	Software Development Workflow: the DLP SDLC skeleton skills, YAML pipeline contracts, EDD iteration with Python evaluations	Mon–Fri full days
3	Model Invocation, Adversarial Agents, and Bounded Agents: earning each step of the automation spectrum	Half Mon + Tue–Thu full + Half Fri
4	Coaching and Delivery: player-coach techniques, artifact standards, readiness gate	Mon–Fri full days

Timing:

- Weeks 1 & 3: Monday sessions begin at **1:00 PM** (travel morning); Friday sessions end at **12:00 PM** (travel afternoon)
- Weeks 2 & 4: All days run **9:00 AM – 5:00 PM** with a 1-hour lunch break

Stretch practice: `Resources/Katas/` contains 14 optional katas (Requirements + Expected Outcomes pairs) for candidates who finish exercises early or want evening reps. They are practice substrate, not required deliverables.

© 2026 Improving

CERTIFICATION GATE (END OF WEEK 4)

To be cleared for DLP engagement, the Embedded Engineer candidate must submit:

- [ ] Manual test files (Input + Expected Output, the EC) for at least 3 skills, written before each skill was built
- [ ] At least one committed skill with `disable-model-invocation: true` that is in active daily use in a real workflow
- [ ] One Python evaluation script with deterministic checks and at least one LLM judge, showing a baseline plus 3+ iterations with documented hypotheses for at least one skill
- [ ] Demonstrated (not merely explained) ability to direct and review AI-written evaluation code
- [ ] Evidence that every instruction in submitted skills is load-bearing (remove-and-test check driven by the evaluation script)
- [ ] One adversarial agent prompt with documented findings from runs against real skill outputs
- [ ] One skill either graduated to model invocation (all three graduation criteria met, with invocation test cases) or accompanied by a written gap specification
- [ ] One bounded agent run: a multi-step pipeline executed in a single IDE session with evaluation at key steps
- [ ] A brief coaching session recording or observation (15–20 min) demonstrating player-coach technique
- [ ] Written self-assessment against the DLP Completion criteria (Craft, Evaluate, Iterate, Adopt)

© 2026 Improving

WEEKLY FILES

- Week-1: Foundations. EDD, Prompt/Harness/Model, RTCC, Social Media Content Creation skills
- Week-2: Software Development Workflow. DLP skeleton skills, YAML pipeline contracts, Python evaluation iteration
- Week-3: Model Invocation, Adversarial Agents, and Bounded Agents
- Week-4: Coaching Readiness and Certification Gate

The `PDFs/` directory contains generated PDF versions of these files (via `generate_pdfs.py`). They are derived artifacts: regenerate them after any content change, never edit them directly.

© 2026 Improving